DAUTH PLATFORM

Enterprise Identity & Cryptographic Intent Authorization Platform

Comprehensive Architectural Specification, Security Proofs & Technical Reference Manual

Platform Version:	v2.4.0-Production-Ready
Security Paradigm:	Zero-Trust Hardware Enclave (WebAuthn / WYSIWYS)
Ledger Architecture:	SHA-256 Hash-Chained Append-Only Audit Log
Primary Repositories:	D2Twins Multi-Tenant Workspace (`backend` & `frontend`)
Target Audience:	Hackathon Judges, Technical Mentors, Security Auditors
Date of Issue:	August 2026

Executive Summary

Modern cloud applications suffer from a fundamental security flaw: **Authentication Token Hijacking**. Once a user authenticates via traditional single-sign-on (SSO) or credentials, an issued Bearer JWT grants total administrative authority over high-risk actions. **DAuth solves this by separating Identity from Intent**. DAuth acts as an enterprise approval platform (akin to Okta/Auth0 combined with a cryptographic policy engine) that requires hardware-backed biometric verification (WYSIWYS — What You See Is What You Sign) for every sensitive transaction, committing signed immutable evidence to a SHA-256 hash-chained ledger.

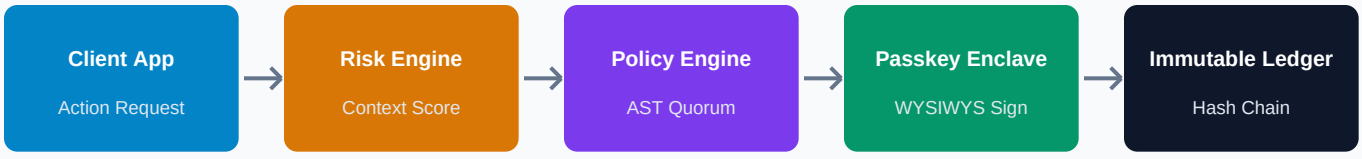


Figure 1.1: Complete End-to-End Execution Pipeline (Zero-Trust Intent Flow)

1. System Architecture & High-Level Execution Pipeline

The DAuth platform is architected around a 6-layer pipeline that ensures every high-risk business transaction is cryptographically non-repudiable and subject to real-time risk and policy evaluation.

Layer	Component	Primary Responsibility & Technical Mechanics
1. Client Apps	Banking, DevOps, Exam Gate	Submits canonical payload JSON (`actionType`, `amount`, `beneficiary`) to `/api/v1/approvals`.
2. Risk Engine	Adaptive Risk Assessor	Evaluates contextual telemetry ($0-100$ score) based on device trust, account age, off-hours access, and value.
3. Policy Engine	AST Rule Evaluator	Parses data-driven JSON rulesets (`ROLE_THRESHOLD`, `SECURITY_OFFICER_VETO`, `M-of-N` quorums).
4. Hardware Enclave	WebAuthn / Passkeys	Prompts physical TouchID/FaceID enclave to generate an ECDSA signature over `payloadHash`.
5. Immutable Ledger	Hash-Chained Audit Log	Appends immutable `LedgerEvent` with sequence ordering and previous hash linkage (H_n).
6. Projection Worker	State Sync & Sockets	Asynchronously recalculates derived state (`APPROVED`/`REJECTED`) and dispatches Socket.IO events.

Core Security Invariants & Principles

1. Intent $\neq$ Identity: Active JWT sessions prove user identity, but never authorize high-risk transactions. Every high-risk action requires a explicit hardware-enclave signature.

2. WYSIWYS (What You See Is What You Sign): The signed challenge is strict SHA-256 hash of the exact payload shown to the user on their screen.

3. Deny Overrides / Veto Precedence: A single authorized veto (e.g. Security Officer) instantly halts an approval workflow regardless of positive votes.

4. One Signer = One Vote: Enforced by database unique constraint `@@unique([requestId, userId])` on `ApprovalDecision`.

2. Domain Model & Prisma PostgreSQL Database Architecture

DAuth implements Domain-Driven Design (DDD). The database schema strictly reflects domain invariants. The **Ledger is the absolute source of truth**; approval requests and application statuses are mere projections that can be completely recalculated from the ledger at any point.

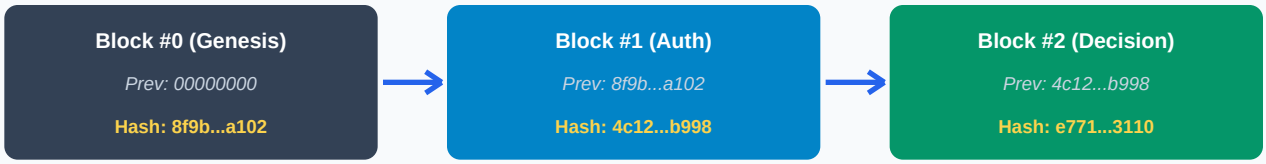
Entity	Table Name	Key Attributes & Relations

Organization	`organizations`	Tenant container (`id`, `name`, `industry`, `protectedApplications`). Has many Users, Apps, Policies.
User	`users`	Identity record (`email`, `password` hash, `publicKey`, `organizationId`). Linked to Devices, Roles.
Device	`devices`	Trusted hardware (`deviceFingerprint`, `publicKey`, `trustLevel`: TRUSTED|UNTRUSTED|REVOKED).
Application	`applications`	Registered OAuth client (`clientId`, `clientSecret` hash, `redirectUrls`, `approvalPolicyId`).
Policy	`policies`	JSON-driven ruleset (`version`, `policyJson`: AST rule tree). Defines required quorums.
ApprovalRequest	`approval_requests`	Business intent (`canonicalPayload`, `payloadHash`, `status`: PENDING|APPROVED|REJECTED).
ApprovalDecision	`approval_decisions`	Individual signature (`signature`, `signedHash`, `timestamp`). Unique on `[requestId, userId]`.
LedgerEvent	`ledger_events`	**Append-only** (`sequence`, `previousHash`, `currentHash`, `nonce`, `signature`). Unmodifiable.
AuditLog	`audit_logs`	Immutable security logs (`eventType`, `actorId`, `ipAddress`, `userAgent`, `metadata`).

3. Cryptographic Security & Hash-Chained Immutable Ledger

DAuth guarantees non-repudiation and insider-threat resistance using hardware WebAuthn credentials and a cryptographically chained ledger.

Figure 3.1: SHA-256 Hash Chain Linking & Cryptographic Non-Repudiation Architecture



Mathematical Proof of Hash-Chain Integrity

Every write operation to the ledger executes within a database transaction enforcing strict row-level locks on sequence numbers. For event n, the current hash H_n is generated as follows:

```
H_0 = SHA-256("GENESIS" || EventData_0 || Nonce_0)
H_n = SHA-256(H_{n-1} || EventData_n || Nonce_n || Signature_n)
```

Tamper Detection Proof: *If a rogue database administrator (DBA) modifies the payload of transaction record k (where $k < n$), the recomputed hash $H'_k \neq H_k$. Because H_{k+1} depends directly on H_k, the entire downstream chain $H_{k+1} \dots H_n$ is invalidated. The DAuth `/api/v1/ledger/verify` endpoint recomputes the chain from H_0 and flags any mismatch instantly as a `SECURITY_VIOLATION`.*

WYSIWYS WebAuthn Serialization Protocol

To prevent payload tampering between the client browser and backend, DAuth forces canonical JSON sorting prior to hashing:

```
1. Sort object keys alphabetically recursively.
2. Serialize to UTF-8 JSON byte string (no whitespace).
3. Compute binary SHA-256 digest: payloadHash = Digest(CanonicalJSON).
4. Pass payloadHash directly as the WebAuthn challenge to `navigator.credentials.get()`.
5. Hardware enclave signs challenge inside AuthenticatorData + ClientDataJSON.
```

4. Adaptive Risk Engine & AST Policy Quorum Engine

The DAuth engine evaluates real-time risk dynamically and enforces data-driven policies stored as Abstract Syntax Trees (AST) in JSON.

Risk Scoring Mechanics (0 – 100 Scale)

Risk Factor	Weight	Evaluation Criteria & Description
Base Action Score	+5 to +30	Action prefix scoring (Delete/Destroy = 30, Transfer = 20, Deploy = 15).
Device Trust Level	+20 to +50	Unregistered device (+30), Untrusted (+20), Revoked (+50).
Account Age	+15	Account created less than 7 days ago.
Recent Failures	+20 to +40	Failed login attempts in the past 60 minutes ($20 \times n$, max 40).
Off-Hours Access	+10	Access outside window (8:00 AM – 8:00 PM local time).
High Value Amount	+20	Transaction amount threshold exceeds ■10,000 / $10,000.

AST Policy Structure Example (JSON Rule Tree)

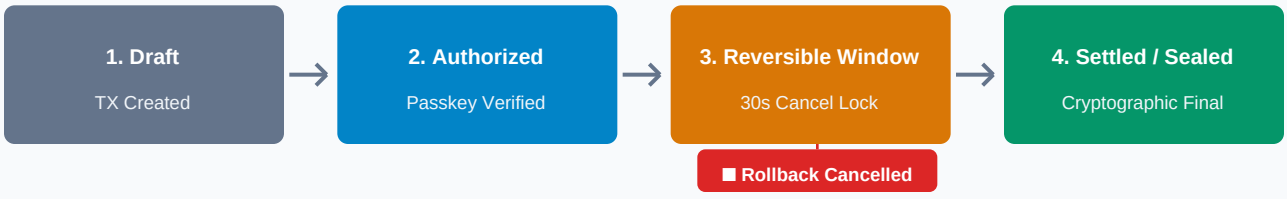
```
{ "version": 1, "expirationMinutes": 60, "approvalRule": { "rule": "ANY", "children": [ { "rule": "ROLE_THRESHOLD", "role": "DEVELOPER", "count": 3 }, { "rule": "ROLE_THRESHOLD", "role": "TEAM_LEAD", "count": 1 } ] }, "vetoRule": { "rule": "ROLE_REJECT", "role": "SECURITY_OFFICER" } }
```

5. Demo Scenarios & Real vs. Simulated Architectural Analysis

To provide 100% transparency for hackathon evaluation and technical mentors, the table below provides a detailed breakdown of backend DB execution vs. frontend UI simulations across all four demo scenarios.

Demo Scenario	Real Backend & DB Execution (Option A)	Simulated / Short-Circuited UI Elements (Option B)
1. NetBanking RTGS Transfer	• Full `/api/v1/approvals` creation in Postgres. • `@simplewebauthn` cryptographic verification. • SHA-256 hash-chained ledger event appended. • Branch Manager decision & policy evaluation.	• Beneficiary account details (HDFC/Sharma Real Estate) are pre-filled in frontend UI state.
2. DevOps Production Release	• First Junior Engineer vote is fully written to DB. • AST Policy Engine evaluates quorum requirement. • Backend genuinely enforces single vote per session (`409 Conflict`).	• Frontend catches `409 AlreadyVotedError` for Junior B & C and generates placeholder ledger cards to demonstrate 3-person quorum on 1 device.
3. University Exam Security Gate	• Live nonce fetched from `/api/v1/approvals/nonce`. • Offline payload hash signed via Passkey WebAuthn. • Submitted via `/api/v1/approvals/offline-submit` into Postgres.	• Simulated 503 network retry loop trigger to showcase client-side resilience and re-sync.
4. Quick Transfer Reversal	• Passkey signature prompt via WebAuthn hook. • Contextual risk warning trigger.	• Reversible 30s settlement timer is pure frontend `setInterval` state. • Post-settlement compensating reversal is a UI workflow walkthrough.

Figure 2.1: Reversible Settlement Window & Compensating Reversal State Transition



6. Cryptographic Recovery Hierarchy (No Vulnerable Email Links)

Email Magic Links represent a single point of failure in modern identity platforms due to phishing, session theft, and mailbox compromise. DAuth completely eliminates Magic Links in favor of a 3-Tier Cryptographic Recovery Hierarchy:

Tier	Recovery Mechanism	Security Guarantee & Workflow
Tier 1	Hardware Recovery Key	High-entropy 32-character offline key generated at enrollment. Never stored unhashed on server.
Tier 2	Trusted Secondary Device	Out-of-band cryptographic approval prompt pushed to an existing enrolled device (e.g. iPad/iPhone).
Tier 3	Organization Administrator Quorum	Requires M-of-N organization admins (e.g. 2 of 3) to cryptographically sign key rotation.

7. Comprehensive REST API Endpoint Reference

Method & Endpoint	Module	Description & Request Body
`POST /api/v1/auth/register/options`	Auth	Generates WebAuthn registration options (`rpID`, `userID`, `challenge`).
`POST /api/v1/auth/register/verify`	Auth	Verifies WebAuthn credential, creates `User` & `Device` records.
`POST /api/v1/auth/login/options`	Auth	Fetches WebAuthn login challenge for registered user devices.
`POST /api/v1/auth/login/verify`	Auth	Validates passkey assertion and issues JWT Access & Refresh tokens.
`POST /api/v1/approvals`	Approvals	Submits new intent (`canonicalPayload`). Evaluates risk & creates request.
`POST /api/v1/approvals/:id/decide`	Approvals	Submits signed decision. Evaluates AST policy and writes to Ledger.
`GET /api/v1/approvals/nonce`	Approvals	Fetches single-use cryptographic nonce for offline exam signatures.
`POST /api/v1/approvals/offline-submit`	Approvals	Processes queued offline signatures with nonce validation.
`GET /api/v1/ledger/:txId`	Ledger	Fetches sequence of immutable ledger events for a transaction.
`POST /api/v1/ledger/verify`	Ledger	Recomputes SHA-256 chain $H_0 \dots H_n$ to verify zero tampering.

End of Official Technical Specification — DAuth Platform